

11 — Expression Indexes (Functional Indexes)

"Index the transformation, not the column — when your query transforms before comparing."

Table of Contents

1. [Learning Objectives](#)
 2. [What is an Expression Index](#)
 3. [Why Functions Break Regular Index Usage](#)
 4. [Case-Insensitive Search](#)
 5. [Date/Time Expression Indexes](#)
 6. [JSON Expression Indexes](#)
 7. [Computed Column Indexes](#)
 8. [Immutability Requirement](#)
 9. [Expression Indexes with Partial Predicates](#)
 10. [Creating Expression Indexes](#)
 11. [EXPLAIN Output](#)
 12. [Expression Index Internals](#)
 13. [Collation and Expression Indexes](#)
 14. [Common Mistakes](#)
 15. [Best Practices](#)
 16. [Performance Considerations](#)
 17. [Interview Questions & Answers](#)
 18. [Exercises with Solutions](#)
 19. [Production Scenarios](#)
 20. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain why applying a function to an indexed column causes index bypass
 - Create expression indexes that match the function applied in queries
 - Design case-insensitive search using `lower()` expression indexes
 - Use `date_trunc()` and other date functions in expression indexes
 - Extract values from JSONB using expression indexes
 - Understand the immutability requirement for expression index functions
-

What is an Expression Index

An **expression index** (also called a functional index) stores the result of a function or expression applied to one or more columns, rather than the raw column values.

```
-- Regular index: stores raw email values
CREATE INDEX ON users(email);

-- Expression index: stores lower(email) values
CREATE INDEX ON users(lower(email));
```

The planner will use the expression index when the query's WHERE clause uses the same expression:

```
-- Uses the lower(email) expression index:
SELECT * FROM users WHERE lower(email) = 'alice@example.com';

-- Does NOT use the lower(email) index:
SELECT * FROM users WHERE email = 'Alice@Example.com';
```

Why Functions Break Regular Index Usage

A B-tree index on column `c` stores values in sorted order: `['alice', 'Bob', 'CAROL']`.

If a query applies `lower(c)`, it needs values in order: `['alice', 'bob', 'carol']`. These are different sorted orders — the index on raw `c` cannot be navigated to find `lower(c) = 'bob'` efficiently, because 'Bob' and 'BOB' are at different positions in the raw-value sorted order.

Raw index on email (case-sensitive sort):

```
'Alice@ex.com' → (0,1)
'BOB@ex.com'   → (0,3)
'alice@ex.com' → (0,2) ← 'a' < 'A' in default collation? Depends on OS!
'bob@ex.com'   → (0,4)
'carol@ex.com' → (0,5)
```

Query: `WHERE lower(email) = 'alice@ex.com'`

- Must find ALL entries where `lower(value) = 'alice@ex.com'`
- 'Alice@ex.com' and 'alice@ex.com' are NOT adjacent in the raw index
- Cannot use a range scan → full index scan or seq scan

Expression index on `lower(email)`:

```
'alice@ex.com' → [(0,1), (0,2)] ← both 'Alice' and 'alice' stored under 'alice'
'bob@ex.com'   → [(0,3), (0,4)]
'carol@ex.com' → (0,5)
```

Query: `WHERE lower(email) = 'alice@ex.com'`

- Navigate tree to 'alice@ex.com' → return ctids → fetch heap
- Efficient $O(\log n)$ lookup!

Case-Insensitive Search

The most common use case for expression indexes.

`lower()` / `upper()` for Case-Insensitive Equality

```
-- Create index
CREATE INDEX CONCURRENTLY idx_users_lower_email ON users(lower(email));
CREATE UNIQUE INDEX idx_users_lower_email_unique ON users(lower(email)); -- unique too!

-- Queries that use the index:
```

```

SELECT * FROM users WHERE lower(email) = 'alice@example.com';
SELECT * FROM users WHERE lower(email) = lower('ALICE@EXAMPLE.COM');

-- Queries that do NOT use the index:
SELECT * FROM users WHERE email = 'alice@example.com';
SELECT * FROM users WHERE email ILIKE 'alice@example.com';
-- Note: ILIKE does NOT use lower() index – use pg_trgm for ILIKE

```

ILIKE Pattern Matching (needs pg_trgm)

```

-- lower() index does NOT help ILIKE with wildcards
-- Use pg_trgm for ILIKE:
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX CONCURRENTLY idx_users_name_trgm ON users USING GIN (name
gin_trgm_ops);
SELECT * FROM users WHERE name ILIKE '%john%';

```

citext Extension (Alternative to lower() Indexes)

```

CREATE EXTENSION IF NOT EXISTS citext;
ALTER TABLE users ALTER COLUMN email TYPE citext;
-- Now all comparisons on email are case-insensitive automatically
-- Regular B-tree index on email now handles case-insensitive queries
CREATE INDEX ON users(email); -- works case-insensitively with citext

```

Date/Time Expression Indexes

Useful for bucketing timestamps and querying by date/time parts.

```

-- Index on date part only (ignoring time)
CREATE INDEX CONCURRENTLY idx_orders_date ON orders(DATE(created_at));
SELECT * FROM orders WHERE DATE(created_at) = '2024-06-15';

-- Index on year+month (monthly partitioning without actual partitioning)
CREATE INDEX CONCURRENTLY idx_orders_yearmonth
ON orders(date_trunc('month', created_at));
SELECT * FROM orders WHERE date_trunc('month', created_at) = '2024-06-
01'::timestamp;

-- Index for day of week analysis
CREATE INDEX CONCURRENTLY idx_orders_dow ON orders(EXTRACT(DOW FROM created_at));
SELECT * FROM orders WHERE EXTRACT(DOW FROM created_at) = 1; -- Mondays

-- Index for hour of day
CREATE INDEX CONCURRENTLY idx_events_hour ON events(EXTRACT(HOUR FROM event_time));
SELECT count(*), EXTRACT(HOUR FROM event_time) AS hour FROM events

```

```
WHERE EXTRACT(HOUR FROM event_time) BETWEEN 9 AND 17
GROUP BY hour ORDER BY hour;
```

Important: AT TIME ZONE

```
-- If users are in different timezones and queries use a specific timezone:
CREATE INDEX CONCURRENTLY idx_orders_local_date
ON orders(DATE(created_at AT TIME ZONE 'America/New_York'));

SELECT * FROM orders
WHERE DATE(created_at AT TIME ZONE 'America/New_York') = '2024-06-15';
```

JSON Expression Indexes

Extract specific JSON fields for efficient lookups.

```
-- Table with JSONB metadata column
CREATE TABLE events (
  id BIGSERIAL PRIMARY KEY,
  created_at TIMESTAMPTZ DEFAULT now(),
  data JSONB
);

-- Index on specific JSON field (extracts as text)
CREATE INDEX CONCURRENTLY idx_events_user_id
ON events((data->>'user_id'));

-- Index on specific JSON field (extracts as integer for numeric comparisons)
CREATE INDEX CONCURRENTLY idx_events_user_id_int
ON events(((data->>'user_id')::INTEGER));

-- Index on nested JSON field
CREATE INDEX CONCURRENTLY idx_events_device_type
ON events((data->'device'->>'type'));

-- Queries that use these indexes:
SELECT * FROM events WHERE data->>'user_id' = '42';
SELECT * FROM events WHERE (data->>'user_id')::INTEGER = 42;
SELECT * FROM events WHERE data->'device'->>'type' = 'mobile';
```

When to Use Expression Index vs GIN on JSONB

Situation	Use
Querying specific fields by exact value	Expression index ((data->>'field'))
Querying multiple fields with containment	GIN index (data jsonb_ops)
Querying nested paths with jsonpath	GIN index with @@

High cardinality field, equality only	Expression index (smaller)
Low cardinality or multi-field queries	GIN index (more flexible)

```
-- Expression index: best for this specific query pattern
CREATE INDEX ON events((data->>'event_type'));
SELECT * FROM events WHERE data->>'event_type' = 'purchase';

-- GIN index: better when query patterns vary
CREATE INDEX ON events USING GIN (data);
SELECT * FROM events WHERE data @> '{"event_type": "purchase", "amount": 99.99}';
```

Computed Column Indexes

Expression indexes can precompute values that would otherwise require CPU work on every row.

```
-- Full name search (first + last)
CREATE INDEX CONCURRENTLY idx_users_full_name
ON users(lower(first_name || ' ' || last_name));

SELECT * FROM users
WHERE lower(first_name || ' ' || last_name) = 'john doe';

-- MD5 hash of email for anonymized logging
CREATE INDEX CONCURRENTLY idx_users_email_hash
ON users(md5(email));
SELECT user_id FROM users WHERE md5(email) = md5('alice@example.com');

-- Computed numeric value
CREATE INDEX CONCURRENTLY idx_products_discounted_price
ON products((price * (1 - discount_pct / 100)));
SELECT * FROM products WHERE (price * (1 - discount_pct / 100)) < 50.00;

-- String length
CREATE INDEX CONCURRENTLY idx_posts_title_length
ON posts(length(title));
SELECT * FROM posts WHERE length(title) > 100;

-- Boolean expression
CREATE INDEX CONCURRENTLY idx_orders_large
ON orders((total_amount > 1000));
-- Note: index stores TRUE/FALSE – rarely useful alone, better as partial index
```

Immutability Requirement

Expression index functions **must be immutable** — they must always return the same result for the same inputs, regardless of when they are called.

Function Volatility Categories

Volatility	Description	Can Use in Expression Index?
IMMUTABLE	Same output for same input, always	Yes
STABLE	Same output per statement (may use session state)	No
VOLATILE	May return different output each call	No

Checking Function Volatility

```
SELECT proname, provolatile
FROM pg_proc
WHERE proname IN ('lower', 'upper', 'date_trunc', 'now', 'random',
'clock_timestamp');
-- provolatile: 'i' = immutable, 's' = stable, 'v' = volatile

-- lower: immutable ✓
-- upper: immutable ✓
-- date_trunc(text, timestamp): immutable ✓
-- date_trunc(text, timestamptz): stable ✗ (depends on timezone setting!)
-- now(): stable ✗
-- random(): volatile ✗
```

The timestamptz Problem

```
-- WRONG: date_trunc on timestamptz is STABLE (depends on timezone GUC)
CREATE INDEX ON orders(date_trunc('month', created_at)); -- created_at is
TIMESTAMPTZ
-- ERROR: functions in index expression must be marked IMMUTABLE

-- WORKAROUND 1: Cast to timestamp (local) first
CREATE INDEX ON orders(date_trunc('month', created_at::timestamp));
-- Converts to local timezone at index build time – may be wrong for timezone-aware
queries

-- WORKAROUND 2: Use AT TIME ZONE with a literal timezone
CREATE INDEX ON orders(date_trunc('month', (created_at AT TIME ZONE
'UTC')::timestamp));
-- AT TIME ZONE with a string literal is immutable

-- WORKAROUND 3: Store date separately
ALTER TABLE orders ADD COLUMN created_date DATE GENERATED ALWAYS AS
(created_at::date) STORED;
CREATE INDEX ON orders(created_date);
```

Custom Immutable Wrapper

```
-- Make an immutable version of a function that's normally STABLE
CREATE OR REPLACE FUNCTION immutable_date_trunc(text, timestampz)
RETURNS timestamp
LANGUAGE SQL IMMUTABLE STRICT AS $$
    SELECT date_trunc($1, $2 AT TIME ZONE 'UTC')::timestamp;
$$;

CREATE INDEX ON orders(immutable_date_trunc('month', created_at));
```

Expression Indexes with Partial Predicates

Combining expression and partial indexes.

```
-- Case-insensitive email index, only for active users
CREATE INDEX CONCURRENTLY idx_active_users_email
ON users(lower(email))
WHERE is_active = TRUE AND deleted_at IS NULL;

-- Query:
SELECT id FROM users
WHERE lower(email) = 'alice@example.com'
    AND is_active = TRUE
    AND deleted_at IS NULL;

-- Month-level date index for recent orders only
CREATE INDEX CONCURRENTLY idx_recent_orders_month
ON orders(date_trunc('month', created_at))
WHERE created_at > '2023-01-01';
```

Creating Expression Indexes

```
-- Syntax: expression must be in parentheses
CREATE INDEX name ON table((expression));
-- or:
CREATE INDEX name ON table(function(column));

-- lower() for case-insensitive search
CREATE INDEX CONCURRENTLY idx_products_lower_name
ON products(lower(name));

-- Extract from JSONB
CREATE INDEX CONCURRENTLY idx_events_session
ON events((data->>'session_id'));

-- Composite with expression
CREATE INDEX CONCURRENTLY idx_users_tenant_lower_email
ON users(tenant_id, lower(email));
```

```
-- Expression as first column, regular column as second
CREATE INDEX CONCURRENTLY idx_orders_month_customer
ON orders(date_trunc('month', created_at), customer_id);

-- Unique expression index
CREATE UNIQUE INDEX idx_users_unique_lower_email
ON users(lower(email));
```

EXPLAIN Output

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, name FROM users WHERE lower(email) = 'alice@example.com';
-- Index: users(lower(email)) INCLUDE (name)
```

```
Index Scan using idx_users_lower_email on users
  (cost=0.43..8.45 rows=1 width=30)
  (actual time=0.034..0.034 rows=1 loops=1)
  Index Cond: (lower((email)::text) = 'alice@example.com'::text)
  Buffers: shared hit=4
Planning Time: 0.189 ms
Execution Time: 0.056 ms
```

Note: `Index Cond` shows the expression `lower((email)::text)` — confirming the expression index is being used.

When expression index is NOT used:

```
EXPLAIN SELECT * FROM users WHERE email = 'alice@example.com';
-- Uses regular email index (if exists) or Seq Scan
-- Does NOT use lower(email) index
```

Expression Index Internals

Expression indexes are stored in `pg_index.indexprs` as a serialized expression tree. When the planner evaluates a query's WHERE clause, it checks whether any expression in the WHERE matches any index's `indexprs`.

```
-- Inspect index expressions
SELECT
  indexname,
  indexdef,
  pg_get_expr(indexprs, indrelid) AS expression
FROM pg_indexes
JOIN pg_index ON pg_indexes.indexname::regclass = pg_index.indexrelid
WHERE tablename = 'users'
AND indexprs IS NOT NULL;
```


Storage

The index stores the **result** of the expression, not the expression itself. For `lower(email)`, the stored value is the lowercased string:

- `'Alice@Example.com'` → stored as `'alice@example.com'`
- `'BOB@TEST.ORG'` → stored as `'bob@test.org'`

This means expression index entries are the same size as storing the function result, not the raw column (which may be larger or smaller).

Collation and Expression Indexes

Collation can affect case-insensitive search behavior.

```
-- Default collation-aware lower():
CREATE INDEX ON users(lower(email));
-- Works for ASCII, but may have issues with locale-specific characters

-- For internationalized case handling, specify collation:
CREATE INDEX ON users(lower(email) COLLATE "C");
-- "C" collation: byte-order comparison, fastest but locale-unaware

-- For proper Unicode case folding:
CREATE INDEX ON users(lower(email) COLLATE "en-US-x-icu");
-- Requires ICU support (PostgreSQL 10+)
```

Common Mistakes

1. Applying a different expression in the query vs the index

```
-- Index on lower(email)
-- Query using upper(email) – NOT the same expression, NOT using the index
WHERE upper(email) = 'ALICE@EXAMPLE.COM'

-- Fix: use the same expression in both index and query
```

2. Using STABLE or VOLATILE functions

```
-- WRONG: now() is STABLE, not IMMUTABLE
CREATE INDEX ON events(date_trunc('day', created_at + now()));
-- ERROR or wrong behavior

-- Fix: use immutable expressions only
```

3. Forgetting the extra parentheses for complex expressions

```
-- WRONG (missing outer parens around complex expression):  
CREATE INDEX ON orders(total * quantity); -- may work for simple expressions  
  
-- BEST PRACTICE (always use parens for clarity):  
CREATE INDEX ON orders((total * quantity));
```

4. Expecting lower() index to work with ILIKE

```
-- lower(email) index does NOT help:  
WHERE email ILIKE 'alice%' -- uses different matching mechanism  
  
-- lower(email) index DOES help:  
WHERE lower(email) LIKE 'alice%' -- equivalent to lower comparison + prefix
```

5. Not updating statistics after creating expression index

```
ANALYZE users; -- update pg_statistic for the new expression index
```

Best Practices

1. **Create the index expression to exactly match the query expression** — even a minor difference (extra cast, different function) prevents the index from being used.
2. **Use IMMUTABLE functions only** — if you need STABLE behavior, create an immutable wrapper function.
3. **Always run ANALYZE after creating expression indexes** so the planner has accurate statistics on the expression values.
4. **Combine with INCLUDE** for covering expression indexes:

```
CREATE INDEX ON users(lower(email)) INCLUDE (id, display_name);
```

5. **For JSONB field extraction**, use expression indexes for frequently-queried specific fields; use GIN for flexible multi-field queries.
6. **Document why you chose an expression index** — it's not obvious why `lower(email)` is indexed instead of `email`.
7. **Test with EXPLAIN** — verify the expression in `Index Cond` exactly matches your query's expression.

Performance Considerations

Build Time and Size

Expression indexes require evaluating the function for every row during index build. Complex expressions (e.g., JSON extraction, string operations) can slow index creation significantly. Use `maintenance_work_mem`

to speed up the sort phase:

```
SET maintenance_work_mem = '1GB';  
CREATE INDEX CONCURRENTLY idx_users_lower_email ON users(lower(email));
```

Update Cost

Every INSERT/UPDATE that affects the indexed column triggers re-evaluation of the expression. For expensive expressions (e.g., regex operations), this can add noticeable write latency.

Statistics Accuracy

The planner uses statistics on expression index values for cost estimation. Run `ANALYZE table_name` after creating an expression index to populate these statistics.

```
-- Check expression index statistics  
SELECT *  
FROM pg_stats  
WHERE tablename = 'users'  
AND attname LIKE '%lower%';
```

Interview Questions & Answers

Q1. Why does a regular B-tree index fail for `WHERE lower(email) = 'foo@bar.com'` even if there's an index on email?

A: A B-tree index on `email` stores values in the raw column's sort order (e.g., 'Alice', 'BOB', 'carol'). A query `WHERE lower(email) = 'foo@bar.com'` would need all entries where `lower()` of the stored value equals the search term. But 'Alice' and 'alice' are stored at different positions in the raw-value sort order — there is no contiguous range of raw email values that correspond to `lower() = 'alice@...'`. The planner would need to scan every index entry and apply `lower()` to check, which is no better than a sequential scan.

Q2. What is the immutability requirement for expression index functions?

A: PostgreSQL requires functions used in index expressions to be IMMUTABLE: they must always return the same output for the same inputs, regardless of any session state, configuration parameters, or the time of evaluation. This is necessary because the index is built once, but the expression must produce consistent results during both index build and query execution. STABLE functions (like `now()`, `current_setting()`) or VOLATILE functions (like `random()`) could produce different results at different times, making the index inconsistent with actual query evaluation.

Q3. How do you index a specific field extracted from a JSONB column?

A: Use an expression index with the `->>` extraction operator: `CREATE INDEX ON events((data->>'user_id'))`. The double parentheses around the expression are required when the expression contains operators. For numeric comparisons, cast the extracted value: `CREATE INDEX ON events(((data->>'amount')::NUMERIC))`. The query must use the exact same expression as the index: `WHERE data->>'user_id' = '42'` or `WHERE (data->>'amount')::NUMERIC > 100`. This is more efficient than a GIN index for single-field, equality-based queries on a specific known field.

Q4. Why is `date_trunc` on `TIMESTAMPTZ` `STABLE` rather than `IMMUTABLE`?

A: `date_trunc('month', timestamptz_value)` truncates to the beginning of the month in the current session's timezone. If the timezone GUC changes between sessions (e.g., one session uses UTC, another uses America/New_York), the same input `timestamptz` would produce different output timestamps. Because the result depends on session configuration, the function is `STABLE` (consistent within one statement) but not `IMMUTABLE`. The workaround is to force a specific timezone: `date_trunc('month', value AT TIME ZONE 'UTC')` which always produces the same result regardless of session timezone.

Q5. Can you create a `UNIQUE` expression index? What does it enforce?

A: Yes. A unique expression index enforces that the expression value is unique across all rows (or rows matching a partial predicate). `CREATE UNIQUE INDEX ON users(lower(email))` prevents two users from having emails that differ only by case, even though the raw email values might differ. This is a common pattern for case-insensitive unique constraints. For example, it would prevent inserting both Alice@Example.com and alice@example.com as separate user emails.

Exercises with Solutions

Exercise 1

A `products` table has a `JSONB metadata` column. A frequent query is: `SELECT * FROM products WHERE metadata->>'brand' = 'Nike' AND metadata->>'category' = 'shoes'` Design the best index strategy.

Solution:

```
-- Option 1: Two expression indexes (if each field is selectively queried alone too)
CREATE INDEX CONCURRENTLY idx_products_brand
ON products((metadata->>'brand'));

CREATE INDEX CONCURRENTLY idx_products_category
ON products((metadata->>'category'));

-- Option 2: GIN index for multi-field containment (more flexible)
CREATE INDEX CONCURRENTLY idx_products_metadata
ON products USING GIN (metadata jsonb_path_ops);
-- Query must change to: WHERE metadata @> '{"brand": "Nike", "category": "shoes"}'

-- Option 3: Composite expression index (if always queried together)
CREATE INDEX CONCURRENTLY idx_products_brand_cat
ON products((metadata->>'brand'), (metadata->>'category'));
-- Query: WHERE metadata->>'brand' = 'Nike' AND metadata->>'category' = 'shoes'
-- Uses composite expression index efficiently

-- Verify:
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM products
WHERE metadata->>'brand' = 'Nike'
AND metadata->>'category' = 'shoes';
```

Production Scenarios

Scenario 1: Global User Authentication

A SaaS platform has users from 50+ countries. Email login must be case-insensitive. The `users` table has 10M rows.

```
-- Case-insensitive unique email constraint + lookup index
CREATE UNIQUE INDEX CONCURRENTLY idx_users_email_ci
ON users(lower(email))
WHERE is_deleted = FALSE;

-- Login query:
SELECT id, password_hash, status
FROM users
WHERE lower(email) = lower($1)
  AND is_deleted = FALSE;

-- Uses partial expression unique index
-- 0.05ms for 10M rows

-- Also works for uniqueness check during registration:
-- INSERT INTO users(email, ...) → constraint checked via idx_users_email_ci
```

Cross-References

- [02 btree indexes.md](#) — B-tree structure
- [08 partial indexes.md](#) — Partial predicates
- [09 composite indexes.md](#) — Composite key design
- [10 covering indexes.md](#) — INCLUDE with expression indexes
- [04 gin indexes.md](#) — GIN for JSONB (alternative to expression indexes)